

00 — อภิธานศัพท์ สัญลักษณ์ และตัวอย่างประจำชุดโน้ต 05-07

ไฟล์นี้คือ **ที่เดียว** ที่คำศัพท์ ตัวอย่าง สัญลักษณ์ คำคงที่ และเลขสมการของโน้ต 05, 06, 07 ถูกนิยาม · โน้ตทั้งสามลิงก์มาที่นี่และ **ไม่มี** **นิยามซ้ำด้วยคำของตัวเอง** — โน้ตมีหน้าที่ *สอน* (ภาพ · ที่มา · การ derive · ความหมาย) ส่วนไฟล์นี้มีหน้าที่ *นิยาม*

อ่านหัวข้อ 0 ก่อนหนึ่งครั้ง จากนั้นใช้เป็นตารางเปิดดู · ทุกบรรทัดโค้ดที่อ้างเป็น file.py:NN ในโฟลเดอร์ tron1-rl-isaacrlab/ (ยกเว้นที่ระบุว่าเป็น IsaacLab/, torch/ หรือ ../ = รากของ study_tron1/) · cfg:NN ย่อจาก limx_rsl_rl_ppo_cfg.py:NN · limx_base_env_cfg.py หมายถึงไฟล์ใน cfg/SF/ เสบอ (PF/WF มีไฟล์ชื่อเดียวกัน แต่เนื้อหาต่างกัน) · :NN เดี่ยว ๆ = ไฟล์เดียวกับที่อ้างล่าสุดก่อนหน้านี้ในแถว/ย่อหน้าเดียวกัน และตรงกับไฟล์จริงแล้ว (2026-09-11) · ตัวเลขทุกตัวในคอลัมน์ "ค่าใน toy" คำนวณจาก ตัวอย่างในหัวข้อ 8 ด้วยสคริปต์เดียวกัน

0. วิธีอ่านตาราง และชนิดของวัตถุ 11 แบบ

ทุกตารางในไฟล์นี้ใช้คอลัมน์ชุดเดียวกัน:

คอลัมน์	บอกอะไร
คำ / สัญลักษณ์	ชื่อไทยก่อน · ชื่ออังกฤษในวงเล็บครั้งแรกที่พบ · สัญลักษณ์เขียนแบบ LaTeX
ชนิด	วัตถุนี้เป็นของประเภทไหน — เลือกจาก 11 แบบข้างล่างเท่านั้น
ความหมาย	หนึ่งประโยค
โค้ด	บรรทัดที่ประกาศ / บรรทัดที่ใช้ · ว่างถ้าเป็นแนวคิดล้วน
ค่าใน toy	ค่าที่ช่อง e0 t0 ของตัวอย่างในหัวข้อ 8 เว้นแต่ระบุช่องอื่น · ถ้าค่าของ Tron1 จริงต่างออกไป จะเขียนต่อท้ายหลังจุดกลาง
ใช้ครั้งแรก	หัวข้อของโน้ต 05 (หลังเรียบเรียงใหม่) ที่ผู้อ่านจะเจอคำนี้ครั้งแรก
สูตร / รรรมเนียม	รูปปิดหนึ่งบรรทัดพร้อมเลขสมการ หรือหมายเหตุเรื่องธรรมเนียมการเขียน

ชนิดของวัตถุ — คำถามแรกของทุกสัญลักษณ์คือ *"มันเป็นของประเภทไหน"* เพราะสัญลักษณ์ ที่หน้าตาคล้ายกันมักเป็นคนละประเภท และประเภทเปลี่ยนไปเรื่อย ๆ ตามสาย:

ฟังก์ชัน → การแจกแจง → เวกเตอร์ → scalar

ชนิด	หมายถึง	ตัวอย่าง
ฟังก์ชัน	รับของเข้าไป คืนของออกมา ยังไม่ได้เรียก	$\pi_\theta, V_\phi, \mu_\theta$
การแจกแจง	ระดัหนึ่งใบ — อีอบเจกต์ Normal ในโค้ด สุ่มได้ ตามความหนาแน่นได้	$\pi_\theta(\cdot s)$
เวกเตอร์	ตัวเลขหลายตัวเรียงกัน มี shape	$\mu(8,), s(42,)$
scalar	ตัวเลขตัวเดียว	$\pi_\theta(a s), L, \rho$
ค่าคงที่	ตัวเลขที่มนุษย์ตั้งใน config ไม่ถูกเรียนรู้	γ, ϵ, lr เริ่มต้น
โค้ด	บรรทัดหรือเมธอดที่ <i>ทำ</i> อะไรสักอย่าง ไม่มีค่าของตัวเอง	<code>.sample()</code> , <code>optimizer.step()</code>
หน่วยนับ	วิธีนับข้อมูล/เวลา	step, episode, iteration
แนวคิด	ไม่มีคำ ไม่มีโค้ด เป็นคำอธิบายพฤติกรรม	on-policy, bootstrapping
เซต	กลุ่มของสมาชิกทั้งหมดที่เป็นไปได้ บอนนี่ ไม่สับสน ไม่คำนวณ	$\mathcal{S}, \mathcal{A}, \Delta(\mathcal{A})$
สมการ	ความสัมพันธ์ที่มีเลขกำกับ — ไม่ใช่ตัวเลข ไม่ใช่โค้ด	Bellman (4), SGD (12), กฎปรับ lr (37)
ตัวดำเนินการ	เครื่องหมายที่บอกว่าจะทำอะไรบางอย่างมาทำอะไรกัน	$\odot, , \sim$

กฎการอ่านที่ใช้ทุกครั้ง — ถ้าสัญลักษณ์ตัวเดียวกันไฟล์ในสองชนิด (เช่น π_θ กับ $\pi_\theta(a | s)$) แปลว่ามัน *ไม่ใช่* ของเดียวกัน แต่เป็นสองจุดบนสายข้างบน — ดูหัวข้อ 7

1. ตัวอย่างและชื่อเฉพาะ

เรียงตามตัวอักษร · ทุกตัวถูกเขียนชื่อเต็มครั้งแรกที่ใช้ในโน้ตแต่ละเล่มด้วย

ตัวอย่าง	ย่อมาจาก	คือ
ADC	Analog-to-Digital Converter	ตัวแปลงสัญญาณแอนะล็อกเป็นตัวเลข — ใช้เป็นอุปมาในโน้ต 05 §2.5
Adam	<i>ไม่ใช่ตัวอย่าง</i> — ชื่อเรียกของ optimizer (มีที่มาจาก adaptive moment estimation)	กฎอัปเดต weight ที่ Tron1 ใช้ (ppo.py:85) · หัวข้อ 5
DDPG · TD3	Deep Deterministic Policy Gradient · Twin Delayed DDPG	อัลกอริทึม policy แบบกำหนดแน่ (deterministic) · โน้ตอ้างเพื่อเทียบเท่านั้น
CLI	Command-Line Interface	บรรทัดคำสั่ง — ที่ที่ <code>--num_envs 2048</code> ถูกพิมพ์กับคำปรีายของ cfg
DP	Dynamic Programming	วิธีแก้ MDP แบบตารางที่รู้แบบจำลองของโลกครม · ทางต้นข้อแรกในโน้ต 05 ภาค 3
EKF · KF	(Extended) Kalman Filter	ตัวประมาณสถานะสาย control · โน้ต 05 §4.4 เทียบกับ encoder
ELU	Exponential Linear Unit	ฟังก์ชันกระตุ้นที่ Tron1 ใช้ทุกชั้นซ่อน (activation="elu", cfg:90) · สมการ (11)
GAE	Generalized Advantage Estimation	วิธีประมาณ advantage ที่ Tron1 ใช้ · สมการ (25)–(26)
GPU · VRAM	Graphics Processing Unit · Video RAM	การ์ดจอและหน่วยความจำบนการ์ด — ที่ที่ทุกอย่างในหัวข้อ 4-5 อาศัยอยู่ตอนแรก

ตัวย่อ	ย่อมาจาก	คือ
IMU	Inertial Measurement Unit	เซนเซอร์วัดความเร่งและอัตราหมุนของลำตัว — ที่มาของ <code>base_ang_vel</code> และ <code>projected_gravity</code>
KL	Kullback–Leibler divergence	ข้อคนสองคน ไม่ใช่ตัวย่อศัพท์ · ตัววัดว่าการแจกแจงสองอันต่างกันแค่ไหน · สมการ (30)
LQR	Linear Quadratic Regulator	ตัวควบคุมสาย <code>control</code> ที่ใช้เทียบกับ <code>critic</code> ในโค้ด 05 §4.3 (อ่านข้ามได้)
lr	learning rate	ก้าวยาวแค่ไหนต่อหนึ่งสเลป · เขียนแทนด้วย α ก็ได้ · หัวข้อ 5
MC	Monte Carlo	ประมาณค่าคาดหวังด้วยการสุ่มพลจิงจอนบ · โค้ด 05 §3.2
MDP · POMDP	(Partially Observable) Markov Decision Process	กรอบคณิตศาสตร์ของปัญหา · "PO" = มองเห็นสถานะไม่ครบ ซึ่งเป็นกรณีของ Tron1
MLP	Multi-Layer Perceptron	โครงข่ายประสาทแบบชั้นซ้อนธรรมดา (<code>nn.Linear</code> สลับกับ <code>activation</code>) — actor, critic, encoder ของ Tron1 ล้วนเป็น MLP
MSE	Mean Squared Error	ค่าเฉลี่ยของกำลังสองของส่วนต่าง — โครงของ <code>value loss</code> และ <code>loss</code> ของ <code>encoder</code>
NN	Neural Network	โครงข่ายประสาทเทียม
ONNX	Open Neural Network Exchange	รูปแบบไฟล์กลางสำหรับ export ในเอนา (<code>play.py:111-121</code>)
PD	Proportional-Derivative controller	ตัวคุมมอเตอร์ชิ้นล่าสุด แปลงมุมเป้าหมายเป็นแรงบิด · สมการ (18)
PDF	Probability Density Function	ฟังก์ชันความหนาแน่นความน่าจะเป็น · สมการ (13)
PPO	Proximal Policy Optimization	อัลกอริทึมที่ Tron1 ใช้แทน "proximal" = คำนวณข้อผิดพลาดของเดิม · Schulman et al. 2017, arXiv:1707.06347
REINFORCE	ชื่อ (Williams 1992) ไม่ใช่ตัวย่อ	policy gradient แบบดั้งเดิมที่ไม่มี <code>critic</code> · โค้ด 05 §4.1
RL	Reinforcement Learning	การเรียนรู้แบบเสริมกำลัง — เรียนจากรางวัล ไม่ใช่จากเฉลย
RNG	Random Number Generator	ตัวสร้างเลขสุ่มของเครื่อง — ที่มาของ ϵ
SAC	Soft Actor-Critic	อัลกอริทึมตระกูล off-policy · อ้างเพื่อเทียบ (<code>.rsample()</code>) เท่านั้น
SDK	Software Development Kit	ชุดโค้ดของ LimX สำหรับคุยกับหุ่นจริง — อยู่นอกขอบเขตโค้ดชุดนี้
PF · WF	PointFoot · WheelFoot	รุ่นเก่าขึ้นของ Tron1 — โค้ดชุดนี้ไม่ใช่ แต่ไฟล์ <code>cfg</code> ของมันยังเข้ากันได้ของ SF (หัวข้อ 0, 6)
SF	SoleFoot	รุ่นเก่าแบบของ Tron1 ที่ task <code>Isaac-Limx-SF-Blind-Flat-v0</code> ใช้ <code>config</code> ของมันคือ <code>SF_TRON1AF1atPPORunnerCfg (limx_rsl_rl_ppo_cfg.py:80)</code>
SGD	Stochastic Gradient Descent	กฎอัปเดตแบบง่ายสุด "ไล่ลงเนิน" · สมการ (12)
SNR	Signal-to-Noise Ratio	อัตราส่วนสัญญาณต่อสัญญาณรบกวน — ใช้อธิบายขนาดก้าวของ Adam
TD	Temporal Difference	"พลจิงก้าวนี้ ต่างจากที่ <code>critic</code> เดาเท่าไร" · สมการ (19)
TRPO	Trust Region Policy Optimization	รุ่นพี่ของ PPO — ยังคับระยะด้วยข้อจำกัด KL แทนการ clip
URDF	Unified Robot Description Format	ไฟล์บรรยายหุ่น (ROS) — ใช้เป็นอุปมาของ MDP ในโค้ด 05 §1.2
VAE	Variational AutoEncoder	สถาปัตยกรรม encoder แบบสุ่ม ซึ่ง repo นี้ ปิดใช้ (<code>is_vae = False</code> , <code>mlp_encoder.py:41</code>)

ตัวย่อที่ **ไม่**ทาง เพราะเป็นชื่อสากลของสิ่งนั้น: `tanh` , `exp` , `log` , Box-Muller, `zigurat`

2. หน่วยนับ

เรียงจากเล็กไปใหญ่ · หน่วยเหล่านี้คือหัววิธีหันตารางเดียวกัน (โค้ด 07 §0)

หน่วย	ชนิด	ความหมาย	โค้ด	ค่าใน toy · Tron1	ใช้ ครั้ง แรก
step (policy step)	หน่วยนับ	หนึ่งรอบการตัดสินใจ 0.02 s = 50 Hz · ข้างในมี <code>physics 4</code> ก้าวย่อยที่ 200 Hz	<code>decimation = 4</code> <code>limx_base_env_cfg.py:497</code> , <code>sim.dt = 0.005</code> :501 · หนึ่ง <code>step = alg.act</code> แล้ว <code>env.step</code> <code>on_policy_runner.py:175-177</code>	1 ช่อง · เหมือนกัน	05 §1.1
transition	หน่วยนับ	ของทีหนึ่ง <code>step</code> ที่วิ่งไปในตาราง: (<i>s</i>, <i>a</i>, <i>r</i>, <i>V</i>, log <i>π_{old}</i>, done)	เก็บโดย <code>add_transitions</code> <code>rollout_storage.py:131-148</code>	1 ช่อง = 6 ค่าที่อัลกอริทึมใช้ (โค้ดเก็บ 12 ช่อง รวม μ, σ :145-146 ที่ KL ต้องใช้) · เหมือนกัน	05 §1.4
episode	หน่วยนับ	ช่วงต่อเนื่องของ <code>step</code> ใน <code>env</code> เดียว จนกว่าจะ <code>done</code>	—	<code>env 1</code> : { <i>t</i> ₀ , <i>t</i> ₁ , <i>t</i> ₂ } จบที่ <i>t</i> ₂ , episode ใหม่เริ่ม <i>t</i> ₃ · ยาวสูงสุด 1000 <code>step</code>	05 §1.4
done	เวกเตอร์ (B,) (B = จำนวน <code>env</code> , สองแถวถัดไป)	ธง 1/0 ว่า <code>step</code> นี้เป็นก้าวสุดท้ายของ <code>episode</code> · เกิดจากสองสาเหตุที่ <i>อัลกอริทึมปฏิบัติต่างกัน</i> (แถวถัดไป)	<code>dones</code> เก็บที่ <code>rollout_storage.py:142</code> · คัดสาย GAE ที่ :194	<i>e</i> 1 <i>t</i> 2 = 1 ที่เหลือ 0	05 §1.4
termination vs truncation	แนวคิด	<i>termination</i> = ลับ (<code>base_contact</code>) — อนาคตถูกตัดจริง · <i>truncation</i> = หมดเวลา (<code>time_out</code>) — อนาคตยังมี แต่เราหยุด	<code>TerminationsCfg</code> <code>limx_base_env_cfg.py:450-454</code> · <code>time_outs</code> ยัง bootstrap : วง γV กลับเข้า <code>reward</code> <code>ppo.py:163-168</code>	toy มีแค่ <code>termination</code> (<i>e</i> 1 <i>t</i> 2)	05 §1.4, §4.5

หน่วย	ชนิด	ความหมาย	โค้ด	ค่าใน toy · Tron1	ใช้ ครั้งแรก
$\text{env} \cdot \mathbf{B}$	หน่วยนับ	หุ่นจำลองหนึ่งตัว · $\mathbf{B}$ = จำนวนที่รันขนานกัน = จำนวนคอลัมน์ของตาราง	num_envs — คำ CLI --num_envs 2048 (../README.md:265 , ../experiments/exp_001_baseline_sf.md:28 — เทียบกับรายนามของ study_tron1/) · ค่าปริยายใน cfg คือ 4096 (limx_base_env_cfg.py:484)	2 · 2048	05 \$1.4
$\mathbf{T}$ จำนวน step ต่อ env	หน่วยนับ	จำนวนแถวของตาราง — เทียบที่ step ต่อ env ก่อนเรียนรู้หนึ่งครั้ง · ในสมการ (2) ตัว $\mathbf{T}$ หมายถึงก้าวสุดท้ายของ episode คณะความหมาย (หัวข้อ 7)	num_steps_per_env = 24 cfg:81	4 · 24	05 \$0.2
rollout	หน่วยนับ	ตารางเต็มหนึ่งใบ = $\mathbf{T} \times \mathbf{B}$ transition ที่เก็บก่อนเรียนรู้ · โป้ตชุดนี้เล็กใช้คำว่า "batch" สำหรับหน่วยนี้	num_steps_per_env = 24 cfg:81	$4 \times 2 = 8 \cdot \mathbf{24} \times \mathbf{2048} = \mathbf{49,152}$	05 \$4.7
mini-batch · MB	หน่วยนับ	ช่องที่สุ่มมา $\mathbf{1/M}$ ของตาราง → หนึ่งใน gradient step	mini_batch_generator rollout_storage.py:222, สุ่มด้วย torch.randperm :230 · num_mini_batches = 4 cfg:99	4 ช่อง · 12,288	05 \$4.7
epoch	หน่วยนับ	เดินผ่านทุกช่องของตารางครบหนึ่งรอบ (สิบครั้งเดียว ต่อ iteration — torch.randperm rollout_storage.py:230 อยู่นอก loop epoch :251 ทุก epoch จึงใช้การสุ่มเดิน)	num_learning_epochs = 5 cfg:98	2 · 5	05 \$4.7
gradient step	หน่วยนับ	หนึ่งครั้งของ zero_grad → backward → clip_grad_norm_ → step	ppo.py:287-290 · ขับด้วย num_updates	$2 \times 2 = 4 \cdot \mathbf{5} \times \mathbf{4} = \mathbf{20}$ ต่อ iteration	05 \$2.2
iteration	หน่วยนับ	เก็บ rollout (ช่วง A) → GAE (ช่วง B) → 20 gradient step (ช่วง C) → ล้างตาราง → เริ่มใหม่	อุปที่ on_policy_runner.py:170 · GAE เรียกที่ :221-225 · update() ppo.py:180 · ล้างที่ :331	1 · 15,000 (cfg:82)	05 \$4.7

3. ปริมาณของโจทก์ — MDP

เรียงตามลำดับที่โป้ต 05 พู

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	ค่าใน toy	ใช้ ครั้งแรก	สูตร / ธรรมเนียม
$\mathcal{S}$ เซตของสถานะ (state space)	เซต	ทุกสถานการณ์ที่หุ่นอาจอยู่ · ตัวใหญ่ = ทั้งเซต	obs 36 ตัวจาก PolicyCfg limx_base_env_cfg.py:130-146 · input จริงของ actor คือ 42 (หัวข้อ 4)	toy ไม่มี $\mathcal{S}$ ชัดเจน — "อะไรก็ตามที่ทำให้ critic พุด 5.0 และ actor พุด 0.20"	05 \$1.2	
$\mathbf{s}_t$ สถานะ ณ step t	เวกเตอร์	สมาชิกหนึ่งตัวของ $\mathcal{S}$		—	05 \$1.3	ตัวห้อย t = step ในตาราง เริ่ม 0 ไม่ทุก iteration
$\mathcal{A}$ เซตของ action (action space)	เซต	ทุกค่าที่สั่งได้ · Tron1: $\mathbf{R}^8$	8 ข้อต่อ: ActionsCfg :116-121 ผ่าน joint_names=[".*"] → solefoot_cfg.py:41-57	$\mathbf{R}$ (1 บิต) · $\mathbf{R}^8$	05 \$1.2	อ่านว่า "การแจกแจงบน $\mathcal{A}$ " ไม่ใช่ "ของ" — หัวข้อ 7
$\mathbf{a}_t$ action ที่สุ่มได้จริง	เวกเตอร์ (8,)	สมาชิกหนึ่งตัวของ $\mathcal{A}$ — ตัวแปรสุ่ม เปลี่ยนทุกครั้งที่ทอย	self.distribution.sample() actor_critic.py:161	0.40	05 \$1.2	ไม่มีหน่วย จนกว่าจะ ×0.25 (สมการ 17)
$P(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})$ transition	การแจกแจง	โลกตอนสมของยังงั — Tron1: PhysX	sim.dt :501, decimation :497	— (ตารางให้ $\mathbf{V}$ มาเลย ๆ)	05 \$1.2	ยึดตั้ง = "เมื่อรู้ว่" — หัวข้อ 7
$R(\mathbf{s}, \mathbf{a})$ ฟังก์ชันรางวัล (reward function)	ฟังก์ชัน	กติกาที่มนุษย์เขียน ให้คะแนนแต่ละ step · เขียนมีวงเล็บเสมอ เพื่อไม่ชนกับ $\mathbf{R}_t$	RewardsCfg limx_base_env_cfg.py:368 — 20 พจน์	—	05 \$1.2	
$\mathbf{r}_t$ reward ที่ได้จริง	scalar	คะแนนของ step เดียว รู้ทันที	rewards rollout_storage.py:141	1.0	05 \$1.1	
γ discount	ค่าคงที่	อนาคตหนึ่งก้าวถัดไปมีค่าที่เปอร์เซ็นต์ของตอนนี	gamma = 0.99 cfg:102 (ค่าปริยายของคลาส ppo.py:51 คือ 0.998 — ถูก cfg กู้)	0.9 · 0.99	05 \$1.2	γ^t · horizon ของ γ เอง $\mathbf{1/(1-\gamma)} = 10 \cdot 100$ step; horizon ที่ GAE ใช้จริงคือ $\mathbf{1/(1-\gamma\lambda)}$ สมการ (34) — แถว λ หัวข้อ 5
Markov property	แนวคิด	อนาคตขึ้นกับสถานะ ปัจจุบันอย่างเดียว ไม่ต้องดูประวัติ	ที่ขาด: base_lin_vel มีใน critic (:183) ไม่มีใน policy obs	toy ถือว่า Markov	05 \$1.3	สมการ (1)
π policy (ทั่วไป) · π^* policy ที่ดีที่สุด	ฟังก์ชัน	กฎเลือก action จากสถานะ — "คำตอบ" ไม่ใช่ "ใจท้ย" · ตาความยถึงตัวที่ทำให้ (2) สูงสุด			05 \$1.2	คำนี้มีสามความหมาย — หัวข้อ 7

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	คำใน toy	ใช้ ครั้ง แรก	สูตร / ธรรมเนียม
objective · $J(\pi), J(\theta)$	ฟังก์ชัน (policy $\rightarrow \mathbb{R}$)	ผลตอบแทนคาดหวังทั้ง episode $\mathbb{E}[\sum_t \gamma^t r_t]$ มองเป็นฟังก์ชันของ policy ($J(\pi)$) หรือของ weight ($J(\theta)$) — ตัวเลขที่ (2) สิ่งให้ทำให้สูงสุด · "สิ่งที่อยากให้สูงขึ้น" ตรงข้ามกับ loss	ไม่มีบรรทัดที่คำนวณ J ตรง ๆ — ลูป 15,000 iteration <code>on_policy_runner.py:170</code> คือการได้เข้าหา	—	05 §1.5	สมการ (2) = "หา π ที่ทำให้ J สูงสุด" (ปัญหาการหาค่าเหมาะที่สุด ไม่ใช่ตัว objective) · S&B เรียก $J(\theta)$ ว่า performance measure (บทที่ 13, §13.2) · คำนี้ซ้ำกับ surrogate ของ PPO — หัวข้อ 7
G_t return	scalar	ผลรวมรางวัลถ่วง γ ตั้งแต่ t จนจบ episode — ข้อเท็จจริง รู้ได้ต่อเมื่อ episode จบ	ไม่มีใน repo — Mean reward (<code>on_policy_runner.py:200-206, :326</code>) คือผลรวมต่อ episode ไม่ถ่วง γ จึงไม่ใช่ G_t · GAE ให้ R_t ไม่ใช่ G_t	env 1 episode แรก: $0.5 + 0.9(1.0) + 0.81(-2.0) = -0.22$	05 §1.7	$G_t = \sum_{k \geq 0} \gamma^k r_{t+k} -$ นิยาม ไม่มีเลขของตัวเอง · ใช้ใน (3), (7)
$V^\pi(s)$ value function	ฟังก์ชัน $\mathcal{S} \rightarrow \mathbb{R}$	ค่าทำนายของ G_t ถ้าเริ่มจาก s แล้วเดินตาม π	<code>self.critic(...)</code> <code>actor_critic.py:170-172</code> คิน (<code>B, 1</code>)	$V(s_0) = 5.0$ (env 0)	05 §1.6	สมการ (3), (7) · ด้วย $\pi =$ "ภายใต้ policy นี้"
$Q^\pi(s, a)$ action-value	ฟังก์ชัน $\mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$	เหมือน V แต่เลือก action แรกไว้ก่อน	ไม่มีใน repo — Tron1 ไม่เคยคำนวณ Q · นิยามไว้เพราะ advantage ต้องใช้	ประมาณหนึ่งก้าว: $1.0 + 0.9(4.5) = 5.05$	05 §1.6	สมการ (5), (6)
Bellman equation	สมการ	V ของวันนี้ = รางวัลก้าวเดียว + $\gamma \cdot V$ ของพรุ่งนี้	ใช้เป็นเข้าใน <code>compute_returns</code> <code>rollout_storage.py:195-199</code>	$5.0 \approx 1.0 + 0.9(4.5) = 5.05$	05 §1.6	สมการ (4)
$A(s, a)$ advantage (นิยาม)	ฟังก์ชัน $\mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$	action นี้ดีกว่าค่าเฉลี่ยของ s เท่าไร · เขียนมีวงเล็บหรือ A_t เสมอ ห้ามเขียน A ลอย ๆ	ไม่มี — ใช้ตัวประมาณ (25)-(26) แทน	$\approx 5.05 - 5.0 = +0.05$ (ประมาณหยาบ)	05 §1.7	$A = Q - V$ สมการ (8)
bootstrapping	แนวคิด	ใช้ค่าทำนายของตัวเอง แทนอนาคตที่ยังไม่รู้	<code>last_values</code> ที่ขอบตาราง <code>rollout_storage.py:190-191</code>	$V_{last} = 3.0$ (env 0), 3.5 (env 1)	05 §1.6	
Monte Carlo vs TD	แนวคิด	MC รอผลจริงจนจบ (ไม่เอนเอียง แต่แกว่ง) · TD ใช้ค่าเดียว + bootstrap (นิ่ง แต่เอนเอียงตาม critic)		env 1 s_0 : MC $-0.22 \cdot$ TD $0.5 + 0.9(2.0) = 2.3 \cdot$ critic 3.0	05 §3.2	
tabular vs function approximator	แนวคิด	ตาราง (หนึ่งช่องต่อสถานะ) vs ฟังก์ชันที่สรุปรวม (NN)	ไม่มีตารางใน repo — RolloutStorage เก็บ transition ไม่ใช่ตารางสถานะ	สถานะของ toy เป็นจำนวนจริง $\rightarrow$ ตารางมีแถวไม่จำกัด	05 §3.1	
on-policy / off-policy	แนวคิด	on: ข้อมูลต้องมาจาก policy ตัวที่กำลังปรับ · off: ข้อมูลมาจาก policy อื่นได้ · PPO เป็น on-policy: ล้างตารางทุก iteration	<code>OnPolicyRunner</code> <code>on_policy_runner.py:46</code> · <code>storage.clear()</code> ppo.py:331		05 §3.3	Sutton & Barto §5.4–5.5
importance sampling	แนวคิด	เอาตัวอย่างจากการแจกแจงหนึ่งไปประมาณค่าเฉลี่ยได้อีกการแจกแจง โดยถ่วงน้ำหนักด้วยอัตราส่วนความหนาแน่น — ที่มาของ ρ			05 §3.3	สมการ (20) · S&B §5.5
explore / exploit · ϵ -greedy	แนวคิด	ลองของใหม่ / ใช้ของที่รู้ ว่าดี · ϵ -greedy (เขียนอีกชละติบ) เป็นวิธีของ Q-learning คนละเรื่องกับ ϵ ในโบต์นี้	Gaussian ทำสองอย่างพร้อมกันด้วย σ		05 §2.4	
deterministic vs stochastic policy	แนวคิด	คิน action ตรง ๆ vs คินการแจกแจงแล้วสุ่ม	<code>act_inference</code> vs <code>act</code> <code>actor_critic.py:159-168</code>		05 §2.1	$a = \mu_\theta(s)$ vs $a \sim \pi_\theta(\cdot s)$

4. ของใน policy และ neural network

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	คำใน toy	ใช้ครั้ง แรก	สูตร / ธรรมเนียม
θ พารามิเตอร์ของ actor	เวกเตอร์ยาว	weight ทั้งหมดที่การเรียนรู้ไปเปลี่ยน = actor $187,272 + \text{logstd } 8 = 187,280$	<code>actor</code> <code>actor_critic.py:69-90</code> , <code>logstd</code> :118	toy ไม่มี NN — μ ให้มาเลย ๆ	05 §2.1	
ϕ พารามิเตอร์ของ critic	เวกเตอร์ยาว	282,625 ตัว	<code>actor_critic.py:92-111</code>	—	05 §2.1	

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	คำใน toy	ใช้ครั้ง แรก	สูตร / สูตรนิยม
ψ พารามิเตอร์ของ encoder	เวกเตอร์ยาว	125,699 ตัว · มี optimizer แยก และถูก <code>.detach()</code> ออกจากกราฟของ loss หลัก	<code>mlp_encoder.py:39</code> , <code>optimizer ppo.py:88-89</code> , <code>loss :313-321</code>	—	05 \$4.4	<code>actor+critic+ℓ = 469,905</code> · รวม <code>encoder = 595,604</code>
node / weight / bias / activation / layer	แนวคิด	หน่วยคำนวณหนึ่งตัว: ผลรวมถ่วงน้ำหนัก + ค่าชดเชย แล้วผ่านฟังก์ชันกระตุ้น	<code>nn.Linear</code> แล้ว <code>activation actor_critic.py:71-90</code> · ขึ้นชื่อ [512, 256, 128] <code>cfg:88-89</code> · ขึ้นสุดท้ายไม่มี <code>activation actor_critic.py:77</code>	—	05 \$2.2	สมการ (10)
ELU	ฟังก์ชัน	$\mathbf{x}$ ถ้า $\mathbf{x} > 0$, $e^{\mathbf{x}} - 1$ ถ้าไม่ · ไม่ "ตาย" ทางลบเหมือน ReLU และไม่อื่นตัวทางบวกเหมือน tanh	<code>get_activation actor_critic.py:175-177</code>	—	05 \$2.2	สมการ (11)
parameter vs hyperparameter	แนวคิด	parameter = ตัวเลขที่ optimizer เขียนกับ (θ, ϕ, ψ) · hyperparameter = ตัวเลขที่มนุษย์ตั้งใน config และไม่ถูกเรียนรู้ (หัวข้อ 6)			05 \$2.2	
o observation ของ policy	เวกเตอร์ (36,)	สิ่งที่เซนเซอร์ให้: <code>ang_vel 3 + gravity 3 + joint_pos 8 + joint_vel 8 + last_action 8 + gait_phase 2 + gait_command 4</code>	<code>PolicyCfg limx_base_env_cfg.py:130-146</code>	—	05 \$1.2	$12 + 3 \times 8 = 36$
c commands	เวกเตอร์ (3,)	ความเร็วเป้าหมาย ($\mathbf{v}_x, \mathbf{v}_y, \omega_x$)		—	05 \$2.2	
h obsHistory	เวกเตอร์ (360,)	o ย้อนหลัง 10 step ต่อกัน · h หมายถึงสิ่งนี้เท่านั้น (หัวข้อ 7)	<code>HistoryObsCfg :154-176</code> (<code>history_length = 10 :175</code>) · <code>obs_history_len = 10</code> <code>cfg:106</code> · $360 = 10 \times 36$ <code>on_policy_runner.py:63</code>	—	05 \$4.4	
z latent — ค่าประมาณ <code>base_lin_vel</code>	เวกเตอร์ (3,)	สิ่งที่ encoder เอาจาก h · เดิมไม่ได้นิยาม z — เปลี่ยนชื่อเป็น z เพราะ loss ของมันคือ MSE กับ <code>base_lin_vel</code> จริง	<code>encode()</code> <code>mlp_encoder.py:95-100</code> (<code>.detach()</code> เมื่อ <code>output_detach = True</code> <code>cfg:109</code>) · <code>loss ppo.py:313-315</code>	—	05 \$4.4	
s input ของ actor	เวกเตอร์ (42,)	[z ; o ; c] ต่อกัน	<code>torch.cat ppo.py:137-139</code>	—	05 \$2.2	$3 + 36 + 3 = 42$
critic input	เวกเตอร์ (230,)	privileged obs 227 + commands 3 · มีของที่หุ่นจริงวัดไม่ได้ (asymmetric actor-critic)	<code>ppo.py:134</code> · <code>base_lin_vel limx_base_env_cfg.py:183</code>	—	05 \$2.1	
π_θ policy ตัวนี้	ฟังก์ชัน $\mathcal{S} \rightarrow \Delta(\mathcal{A})$	รับ s คืนการแจกแจงบน $\mathcal{A}$ — ไม่ได้คืน action · ประกอบด้วย ① NN ② <code>logstd</code> ③ สูตร $\mathcal{N}$ · ไม่มีออบเจกต์เดียวในโค้ดที่เท่ากับมัน (<code>self.actor_critic</code> มี critic ปนอยู่ด้วย)	①+②+③ ใน <code>actor_critic.py:155-157</code>		05 \$2.1	สมการ (9)
$\Delta(\mathcal{A})$ simplex	เซต	เซตของการแจกแจงทั้งหมดบน $\mathcal{A}$ — "ระบัพทุกใบที่วางบน $\mathcal{A}$ ได้"			05 \$2.1	คนละตัวกับ δ_t และ $\Delta\mu$ — หัวข้อ 7
$\mu_\theta(\mathbf{s})$ จุดกลาง (mean)	เวกเตอร์ (8,)	output ของ NN = ค่าที่ actor คิดว่าดีที่สุด · นิพจน์ ค่าบวกใหม่ทุกครั้ง	<code>mean = self.actor(observations)</code> <code>actor_critic.py:156</code>	0.20	05 \$2.1	
ℓ logstd	เวกเตอร์ (8,)	ลูกบิตคูณความกว้างหนึ่งตัวต่อข้อต่อ · ตัวแปร ที่บนอยู่ข้าง ๆ NN ไม่ใช่ข้างใน ไม่รับ input · เปลี่ยนค่าได้ทางเดียวคือ <code>optimizer.step()</code> เขียนกับ · ไม่ใช่ $\mathcal{L}$	<code>nn.Parameter(torch.zeros(num_actions))</code> <code>actor_critic.py:118</code>	$\ln 0.5 = -0.6931$ (คงที่ใน toy) · Tron1 เริ่ม 0	05 \$2.1	$\sigma = e^{\ell}$ สมการ (9)
σ ส่วนเบี่ยงเบนมาตรฐาน (standard deviation)	เวกเตอร์ (8,) broadcast → (B, 8)	ความกว้างของระบัพ · ไม่ใช่กับ s ทุก state ทุก env ใช้ชุดเดียวกัน · ไม่ได้มาจาก critic · ไม่ใช่ความแปรปรวน (σ^2)	<code>torch.exp(self.logstd)</code> <code>actor_critic.py:157</code> · ค่าเฉลี่ยขึ้นล็อก <code>Policy/mean_noise_std</code> <code>on_policy_runner.py:269, :284</code>	0.5 คงที่ · Tron1 เริ่ม 1.0 แล้วลด	05 \$2.1	ใน <code>torch.distributions.Normal</code> ช่องนี้ชื่อ <code>scale</code> · นิพจน์นี้เขียน $\mathcal{N}(\mu, \sigma)$ ดำเราเขียน $\mathcal{N}(\mu, \sigma^2)$ — หัวข้อ 7

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	ค่าใน toy	ใช้ครั้งแรก	สูตร / ธรรมเนียม
$\pi_\theta(\cdot s)$ ะวัง 1 ใน	การแจกแจง	ผลของการใส่ s เข้า π_θ · จุดกลม = ช่องที่ยังไม่เต็ม · สร้างใหม่และทิ้งทุกสแต็ป ไม่มี weight	self.distribution = Normal(mean, ...) actor_critic.py:157 — ร็อบเบอร์ต ไม่ใช้ตัวเลว	$\mathcal{N}(0.20, 0.5)$	05 \$2.1	$\pi_\theta(\cdot s) = \mathcal{N}(\mu_\theta(s), \sigma)$ สมการ (9)
ϵ noise มาตรฐาน	เวกเตอร์ (8,)	เลขสุ่มจาก $\mathcal{N}(0, 1)$ ที่ทำให้ $a \neq \mu \cdot \epsilon$ หมายถึง สิ่งนี้เท่านั้น (หัวข้อ 7)	อ้างใน torch.normal	+0.40	05 \$2.4	$a = \mu + \sigma \odot \epsilon$ สมการ (16)
$\odot$ Hadamard	ตัวดำเนินการ	คูณทีละตัวตรงตำแหน่ง — ในโค้ดคือ $*$			05 \$2.4	ไม่ใช่ dot product
$a \sim \pi_\theta(\cdot s)$ สุ่ม	โค้ด → เวกเตอร์	tilde อ่านว่า "ถูกสุ่มออกมาจาก" · เรียกซ้ำได้ค่าใหม่ — ไม่ใช่ฟังก์ชัน	.sample() actor_critic.py:161 (ได้ no_grad; torch.normal(loc, scale)) · เทียบ .rsample() ที่เขียน $\mu + \sigma \epsilon$ ตรงตัว — Tron1 ไม่ใช้	0.40	05 \$2.4	
z_i ะยะมาตรฐาน	scalar ต่อมิติ	a_i ห่างจาก μ_i ที่เท่าของ $\sigma_i \cdot z$ ตัวเดียวห้ามใช้เรียก latent อีก		0.40	05 \$2.3	$z_i = (a_i - \mu_i)/\sigma_i$ · $\partial \log \pi / \partial \ell_i = z_i^2 - 1$
$\pi_\theta(a s)$ ความหนาแน่น ณ จุด a	scalar	ความสูงของระวัง ณ a · เกิน 1 ได้ เพราะสิ่งที่ต้องเป็น 1 คือพื้นที่ ไม่ใช่ค่าความสูง	.log_prob(a).sum(dim=-1) actor_critic.py:164 (คืน log)	0.7365 · ยอดที่ $\sigma = 0.5$ คือ 0.7979 · ที่ $\sigma = 0.25$ ยอด 1.596	05 \$2.3	สมการ (13)
$\log \pi_\theta(a s)$	scalar ต่อ env (B,)	log ของความหนาแน่น รวมทุกข้อต่อ · เก็บลง buffer เป็น log pids	torch/distributions/normal.py:80-92 แล้ว .sum(dim=-1) actor_critic.py:164 · เก็บ ppo.py:147-149	-0.3058 (1 บิต)	05 \$2.3	สมการ (14), (15)
act vs act_inference	โค้ด	ตอนเรน: สร้างระวังแล้วสุ่ม · ตอน deploy: คืน μ ตรง ๆ ไม่มี σ ไม่มีการสุ่ม	actor_critic.py:159-161 vs :166-168 · get_inference_policy on_policy_runner.py:376		05 \$2.6	deploy $\neq$ " $\sigma = 0$ " — σ ถูกทิ้ง
scale 0.25 · default offset	ค่าคงที่	แปลง action ไร้หน่วยเป็นเรเดียนรอบท่าเริ่มต้น	scale=0.25, use_default_offset=True limx_base_env_cfg.py:119-120 · คุณ-บวกที่ IsaacLab/.../joint_actions.py:134 · offset = default_joint_pos :155-156 = 0.0 ทุกข้อ (solefoot_cfg.py:32-34)	0.40 → 0.10 rad (5.7°)	05 \$2.5	สมการ (17)
PD controller · k_p, k_d	ฟังก์ชัน	แรงบิด = $k_p \times$ ความคลาดเคลื่อน - $k_d \times$ ความเร็ว · ทำงาน 200 Hz	stiffness=45.0, damping=1.5 (บาท) solefoot_cfg.py:50-51, damping=0.8 (ข้อเท้า) :61-62, effort_limit=80 :48,:59 · ส่งเป้าที่ joint_actions.py:160	—	05 \$2.5	สมการ (18)
gradient descent	แนวคิด/สมการ	หมุนลูกบิดส่วนทางความชันของ L	รูปจริงคือ Adam ppo.py:85		05 \$2.2	สมการ (12)
backpropagation · chain rule	แนวคิด	คำนวณ $\partial L / \partial (\text{ทุก ๆ คำนวณ})$ ย้อนกลับตามกราฟที่ forward บันทึกไว้ = loss.backward()	ppo.py:288		05 \$2.2	
autograd	โค้ด	ระบบหาอนุพันธ์อัตโนมัติของ PyTorch — สนแค่ว่าตัวเลขนี้ไหลในนิพจน์ที่นำไปสู่ L โหมด ไม่สนว่าเป็นเลขหรือค่าหรือเปล่า			06 \$3.4	
Jacobian	แนวคิด	เมทริกซ์ของอนุพันธ์ย่อยของฟังก์ชันหลายมิติ · ใช้ตอนเปลี่ยนตัวแปรในความหนาแน่น			06 \$3.4	$ \det \partial \epsilon / \partial a = 1 / \prod_i \sigma_i$
universal approximation	แนวคิด	NN ที่ใหญ่พอประมาณฟังก์ชันต่อเนื่องใดก็ได้ — เหตุผลที่ actor "ควม" ให้ action ตรง ๆ ได้ แต่ RL ต้องการการแจกแจงด้วยเหตุผลอื่น			06 \$3.5.0	
supervised learning	แนวคิด	เรียนจากคู่ (input, เฉลย) · critic เรียนแบบนี้ แต่เฉลยของมันมีตัวมันเองปอนอยู่ (bootstrap)	value loss ppo.py:263-271		06 \$3.4	
logits	แนวคิด	output ดิบก่อน softmax ของ head แบบ categorical · Tron1 ไม่มี — action ต่อเนื่อง			06 \$3.5.0	

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	คำใน toy	ใช้ครั้ง แรก	สูตร / ธรรมเนียม
<code>.detach()</code> · <code>no_grad</code> · <code>nn.Parameter</code> · <code>requires_grad</code> · <code>.grad</code>	โค้ด	ตัดกราฟ / ไม่บันทึกกราฟ / <code>tensor</code> ที่ <code>optimizer</code> เห็น / รงว่าอยากได้ <code>grad</code> / ช่องข้าง ๆ ที่ <code>backward()</code> เติบ	a ถูก <code>detach</code> <code>ppo.py:139</code> · <code>logstd</code> เป็น <code>Parameter</code> <code>actor_critic.py:118</code>		06 \$3.4	<code>.grad</code> เป็นคนละกล่องกับคำ — หัวข้อ 5
reparameterization · <code>.rsample()</code>	แนวคิด	เขียนการสุ่มเป็น $\mu + \sigma \epsilon$ เพื่อให้ <code>gradient</code> ไหลผ่าน $\mathbf{a}$ ได้ · PPO ไม่ต้องใช้ (<code>gradient</code> ไหลผ่าน $\log \pi$) · SAC เลือกใช้เพราะ <code>variance</code> ต่ำกว่า	<code>torch/distributions/normal.py:75-78</code> — ไม่มีใน repo		06 \$3.5.4	

5. ของใน PPO loss และการเรียนรู้

เรียงตาม ลำดับการคำนวณ = `rollout_storage.py:187-207` แล้ว `ppo.py:205-290`

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	คำใน toy	ใช้ ครั้ง แรก	สูตร / ธรรมเนียม
V_{last} bootstrap ที่ขอบตาราง	เวกเตอร์ ($B, 1$)	critic ทำนายจากสถานะหลัง <code>step</code> สุดท้าย เพราะแถว T ไม่มีใครให้ดู	<code>compute_returns(last_values)</code> <code>rollout_storage.py:190-191</code> , เรียกจาก <code>ppo.py:176-178</code> , <code>on_policy_runner.py:221-225</code>	$3.0 \text{ (env 0)} \cdot 3.5 \text{ (env 1)}$	05 \$3.2	
$mask \text{ (} 1 - done_t \text{)}$	เวกเตอร์	ตัดพจน์ bootstrap ที่เมื่อ <code>episode</code> จบที่ช่องนี้	<code>next_is_not_terminal</code> <code>rollout_storage.py:194</code>	1 ทุกช่องยกเว้น <code>e1 t2 = 0</code>	05 \$3.2	
δ_t TD error	scalar ต่อช่อง	ผลจริงที่ตัวนี้ $+ \gamma \times$ ทำนายพรุ่งนี้ – ทำนายวันนี้ = critic พลาดเท่าไร	<code>delta</code> <code>rollout_storage.py:195-199</code>	$+0.05 \cdot e1 \text{ } t2 = -3.00$ (ถ้าไม่มี <code>mask</code> จะได้ $+0.6$ ซึ่งผิด)	05 \$3.2	สมการ (19)
λ GAE <code>lambda</code>	ค่าคงที่	เชื้อ <code>critic</code> แคไหน vs รอผลจริง (0 = TD ล้วน, 1 = MC ล้วน)	<code>lam = 0.95</code> <code>cfg:103</code>	$0.8 \cdot 0.95$	05 \$4.5	$\gamma \lambda = 0.72 \cdot 0.9405$
A_t advantage (ตัวประมาณ GAE, ยังไม่ <code>normalize</code>)	scalar ต่อช่อง	ผลรวมถ่วงน้ำหนัก $(\gamma \lambda)^k$ ของ δ ในอนาคต หยุตที่ขอบ <code>episode</code>	<code>recursion</code> <code>rollout_storage.py:200</code>	$-0.2722 \cdot e1 \text{ } t0 = -2.3272$	05 \$4.5	สมการ (25) พหุคูณ $\equiv$ (26) <code>recursion</code>
R_t return เป้าของ <code>critic</code> (<code>returns</code>)	scalar ต่อช่อง	$A_t + V_t$ — ไม่ใช่ G_t มีคำทำนายของ <code>critic</code> ผนอยู่	<code>self.returns[step]</code> <code>rollout_storage.py:201</code>	4.7278	05 \$4.5	สมการ (27)
$\hat{A}_t$ advantage ที่ <code>normalize</code> แล้ว (<code>advantages</code>)	scalar ต่อช่อง	A ลบค่าเฉลี่ยหารด้วย ส่วนเบี่ยงเบนของทั้งตาราง · <code>std</code> ใช้ $n - 1$ · ตัวที่ <code>loss</code> ใช้จริงคือตัวนี้	<code>rollout_storage.py:204-207</code>	$+0.7975 \cdot \text{mean}(A) = -1.1904$, <code>std = 1.1513 \cdot e1 \text{ } t1 = -0.9290</code>	05 \$4.5	สมการ (28)
<code>policy gradient</code>	สมการ	ทิศที่ต้องขยับ θ เพื่อให้ผลตอบแทนคาดหวังสูงขึ้น = ค่าเฉลี่ยของ $\nabla \log \pi \cdot A$	ไม่มี REINFORCE ใน repo — <code>autograd</code> ผ่าน <code>actor_critic.py:164</code> → <code>ppo.py:288</code> หน้าที่นี่		05 \$4.1	สมการ (22), (23) · S&B §13.2-13.4
$\partial \log \pi / \partial \mu_i$	scalar ต่อมิติ	$\log \pi$ ๖ต่อ μ แคไหน		$(0.40 - 0.20) / 0.25 = 0.80$	05 \$4.1	สมการ (24)
<code>surrogate</code>	แนวคิด	ฟังก์ชันตัวแทนของเป้าหมายจริงที่คำนวณไม่ได้จากข้อมูลเก่า · ความชันตรงกับของจริงเฉพาะแถว ๆ θ_{old} — จึงต้อง <code>clip</code>			05 \$4.6	
ρ_t probability ratio	scalar ต่อช่อง	<code>policy</code> ใหม่ชอบ <code>action</code> เก่านั้นมากขึ้นที่เก่า · $\mathbf{a}$ ตัวเดิมจาก <code>buffer</code> ถูกถามะระวังสองใบ	$\log \pi_{new}$ <code>ppo.py:212-214</code> (หลัง <code>act()</code>) ที่ <code>:205</code> สร้างะวังใหม่แล้วไล่น <code>sample</code> ทั้ง) · <code>ratio</code> <code>:252-254</code>	$1.0356 \text{ (} \mu \text{ ขยับ } +0.05 \text{)} \cdot e0 \text{ } t2 / e1 \text{ } t1 = 1.0779$	05 \$3.3	สมการ (21) · = 1 ที่ <code>gradient step</code> แรก
ϵ ขอบ <code>clip</code>	ค่าคงที่	ρ ขยับได้ไม่เกิน $\pm 20\%$ ต่อ <code>rollout</code> · ϵ หมายถึงสิ่งนี้เท่านั้น	<code>clip_param = 0.2</code> <code>cfg:96</code>	0.2	05 \$4.6	

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	ค่าใน toy	ใช้ ครั้ง แรก	สูตร / ธรรมเนียม
L^{CLIP} surrogate loss	scalar	— $\hat{A}\rho$ กับ — $\hat{A}\text{clip}(\rho)$ เอาตัว ที่ใหญ่กว่า (แยกว่า) แล้วเฉลี่ยทั้ง minibatch · โค้ดเดิม สลับแล้ว เปรียบ เขียน max ของ min ไม่มีสว	ppo.py:256-260	ช่อง e0 t0 = -0.8259 · mb ₀ เฉลี่ย = -0.2958	05 \$4.6	สมการ (29) · clip ปิด gradient เฉพาะทิศที่ถูกและโกลาพอ — โค้ด 07 \$5.3
L_V value loss	scalar	critic ทำนายพลาด จาก R_t เท่าไร (กำลัง สอง) — แบบ clipped: ใช้ตัวที่ใหญ่ กว่าระหว่างพลาดจริง กับพลาดของค่าที่ถูก หั่น	ppo.py:263-271 · use_clipped_value_loss = True cfg:95	e0 t0 (V _{new} 4.5): 0.0519	05 \$4.6	สมการ (32)
H entropy	scalar	ความไม่แน่นอนของ ระยะ · ขึ้นกับ σ อย่าง เดียว ไม่เกี่ยวกับ μ · รวม 8 บิตแล้วเฉลี่ย minibatch	actor_critic.py:151-153, ppo.py:273 · สูตร torch/distributions/normal.py:104- 105	0.7258 ต่อบิต · ×8 = 5.8063	05 \$4.6	สมการ (31) · $\partial H / \partial \ell_i = 1$
c_V, c_H สัมประสิทธิ์	ค่าคงที่	น้ำหนักของ L_V และ H ใน loss รวม	value_loss_coef = 1.0 cfg:94, entropy_coef = 0.01 cfg:97 (ค่าปรัย ของคลาส ppo.py:54 คือ 0.0 — ถูก cfg กับ)	1.0 · 0.01	05 \$4.6	
L loss รวม	scalar ตัวเดียว ทั้งระบบ	เพิ่มวัดความแปร ซึ่ง น้อยยิ่งดี · ไม่ใช่ ℓ · KL ไม่อยู่ที่นี่	ppo.py:274-278	—	05 \$2.2	$L = L^{CLIP} + c_V L_V - c_H H$ สมการ (33)
$\nabla_{\theta} L$ · loss.backward()	เวกเตอร์ ยาว / โค้ด	ความชันของ L เทียบ กับลูกบิตทุกตัว (469,905 ตัวใน optimizer หลัก) · เขียนลงช่อง .grad ไม่แตะค่า	ppo.py:288		05 \$2.2	
max_grad_norm · clip_grad_norm	ค่าคงที่ / โค้ด	ถ้า norm ของ gradient ทั้งก้อนเกิน 1.0 ย่อลงตามสัดส่วน — เขียนกับ .grad ก่อน step() เห็น	ppo.py:289 · cfg:105	1.0	05 \$4.6	คร่อม actor+critic+ ℓ · encoder แยก (ppo.py:319-321)
α · lr learning rate	scalar (ปรับได้)	ก้าวยาวแค่ไหน · hyperparameter — เริ่มจากที่มนุษย์ พิมพ์ แล้ว KL ปรับเอง	learning_rate = 1.0e-3 cfg:100 · ไล่ Adam ppo.py:85	0.1 ใน toy (เพื่อให้เห็นผล) · 1e-3	05 \$2.2	สมการ (12), (37)
SGD	สมการ	$\theta \leftarrow \theta - \alpha \nabla L$ — ก้าวแปรตามขนาด ความชัน	ไม่ได้ใช้จริง — ใช้เป็นรูปอ่านทิศ		05 \$2.2	สมการ (12)
Adam · $\hat{m}, \hat{v}$ · eps	สมการ	SGD ที่หารขนาดความ ชันทั้ง (อัตราส่วน สัญญาณ/สัญญาณ รบกวน) — ก้าวราว α เมื่อความชันชี้ทาง เดิมสม่ำเสมอ น้อย กว่านั้นมากเมื่อเบว · eps = 1e-8 ค่า ปรัย torch	optim.Adam(...) ppo.py:85 · เขียนกับที่ torch/optim/adam.py (param.addcddiv_)		05 \$2.2	สมการ (36)
optimizer.step()	โค้ด	ที่เดียวที่ตัวเลขของ actor, critic, ℓ เปลี่ยนค่าระหว่างทรน · อ่าน .grad แล้ว เขียนกับ · ไม่ได้ คำนวณ gradient	ppo.py:290 · encoder แยก :321		05 \$2.2	
$\partial L / \partial \mu$	scalar ต่อบิต	ทิศที่ μ ต้องขยับ: เข้าหา $\hat{a}$ ที่ $\hat{A} > 0$ หนีจาก $\hat{a}$ ที่ $\hat{A} < 0$	autograd	-0.4955 (บี p)	05 \$4.1	สมการ (24) → โค้ด 07 (G)
$\partial L / \partial \ell$	scalar ต่อบิต	ทิศที่ σ ต้องขยับ: กว้างขึ้นเมื่อค่าดีอยู่ ไกล / ทำแย่อยู่ใกล้ · entropy ดันกว้าง คงที่	autograd	ที่ step แรก (p=1): surrogate +0.6699, entropy -0.01, รวม +0.6599 → ด้วย lr จริง 10^{-3} (ไม่ใช่ 0.1 ของ toy): ℓ -0.6931 → -0.693807, σ → 0.499670	06 \$3.4	สมการ (35)

ค่า / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	ค่าใน toy	ใช้ ครั้ง แรก	สูตร / สูตรนิยม
$KL D_{KL}(\pi_{old} \parallel \pi_{new})$	ฟังก์ชัน (การ แจกแจง, การ แจกแจง) → scalar ≥ 0	ระยะห่างระหว่างฟังก์ชัน ที่ใช้เก็บข้อมูล รอบนี้ แค่ โหนด (ระยะสะสมตั้งแต่ ต้น update() ไม่ใช่ ก้าวล่าสุด) · ไม่ สมมาตร · หน่วย nat · ไม่อยู่ใน loss	รายมิติแล้วบวก 8 มิติ ppo.py:223-231 (มี +1e- 5 ใต้ log) → kl_mean :233 → กฎ lr :236- 244 · ขึ้นชื่อเป็น Policy/mean_kl (คนละ ตัวแปร, :295, on_policy_runner.py:285)	$\Delta\mu = 0.05$: 0.0050 · $\Delta\mu = 0.15$: 0.045	05 \$4.6	สมการ (30) · ถ้า σ เท่ากัน เหลือ $(\Delta\mu)^2/(2\sigma^2)$
desired_kl · schedule	ค่าคงที่	อุณหภูมิเป้าหมายของ thermostat · adaptive = เปิด กฎ	cfg:104, cfg:101	0.01	05 \$4.6	
กฎปรับ lr	สมการ	$KL > 0.02 \rightarrow lr \div 1.5$ (พื้น 1e-5) · $KL < 0.005 \rightarrow lr \times 1.5$ (เพดาน 1e-2) · เช็ก ทุก minibatch (20 ครั้ง/iteration)	ppo.py:236-244	$0.0050 \rightarrow$ คงเดิม · $0.045 \rightarrow \div 1.5$	05 \$4.6	สมการ (37) · จาก 1e-3 ขนเพดานใน 6 ครั้ง ขนพื้นใน 12 ครั้ง
early_stop · anneal_lr	ค่าคงที่	ทั้งคู่ ปิด (ค่าปรีาย) · anneal ถ้าเปิดจะเป็น พื้นเลื้อยภายใน iteration ไม่ใช่ลดตาม เวลา	ppo.py:63-64 · ทั้ง :246-249, :280-284		06 \$3.4	
extra_optimizer · est_learning_rate	โค้ด / ค่า คงที่	optimizer แยกของ encoder · lr ของมัน ไม่ถูก KL และ	ppo.py:61 (ค่าปรีาย 1e-3 ไม่ได้ตั้งใน cfg), :88- 89, .step() :321		06 \$3.4	
encoder loss	scalar	MSE ระหว่าง $\hat{v}$ กับ base_lin_vel จริง (3 ช่องแรกของ critic obs)	ppo.py:313-315		05 \$4.4	

6. ค่าคงที่จาก config

หนึ่งแถวต่อบรรทัดของ block ที่รันจริง SF_TRON1AFlatPPORunnerCfg (limx_rsl_rl_ppo_cfg.py:80-114) · ไฟล์เดียวกันมี block ของหุ่นรุ่นอื่น (PFPPPORunnerCfg :10-38, PF :42-76, WF :119-153) ซึ่ง**ทั้งบรรทัดและค่าบางตัวต่างกัน** (เช่น max_iterations 2000 / 10000, save_interval 200) — อ้างบรรทัดใน block นี้เท่านั้น

บรรทัด	ชื่อ	ค่า Tron1	ค่าใน toy	หมายถึง	อธิบาย ที่
cfg:81	num_steps_per_env	24	4	จำนวนแถวของตาราง (T)	05 \$4.7
cfg:82	max_iterations	15,000	1	จำนวนตารางทั้งหมด	05 \$1.5
cfg:83	save_interval	500	—	บันทึก checkpoint ทุกที่ iteration	05 \$2.6
cfg:87	init_noise_std	1.0	—	ดูเหมือนไม่ถูกใช้ : บรรทัดที่ใช้บันทึกคอมเมนต์ทั้ง (actor_critic.py:117) และ :118 เริ่ม ϵ ที่ 0 → σ = 1.0 อยู่แล้ว	05 \$2.1
cfg:88-89	actor_hidden_dims · critic_hidden_dims	[512, 256, 128]	—	ชั้นซ่อน	05 \$2.2
cfg:90	activation	"elu"	—	ฟังก์ชันกระตุ้น	05 \$2.2
cfg:94	value_loss_coef	1.0	1.0	c_V	05 \$4.6
cfg:95	use_clipped_value_loss	True	True	เปิด value clipping	05 \$4.6
cfg:96	clip_param	0.2	0.2	ϵ — ใช้ทั้ง p clip และ value clip	05 \$4.6
cfg:97	entropy_coef	0.01	0.01	c_H	05 \$4.6
cfg:98	num_learning_epochs	5	2	epoch ต่อ iteration	05 \$4.7
cfg:99	num_mini_batches	4	2	หั่นตารางกี่ก้อน	05 \$4.7
cfg:100	learning_rate	1e-3	0.1 (SGD เพื่อดูแล)	α เริ่มต้น	05 \$2.2

บรรทัด	ชื่อ	ค่า Tron1	ค่าใน toy	หมายถึง	อธิบาย ที่
cfg:101	schedule	"adaptive"	—	เปิดกฎ KL	05 \$4.6
cfg:102	gamma	0.99	0.9	γ	05 \$1.2
cfg:103	lam	0.95	0.8	λ	05 \$4.5
cfg:104	desired_kl	0.01	0.01	เป้าหมาย thermostat	05 \$4.6
cfg:105	max_grad_norm	1.0	—	เพดาน norm ของ gradient	05 \$4.6
cfg:106	obs_history_len	10	—	ความยาวประวัติ $\rightarrow$ 360	05 \$4.4
cfg:109-111	output_detach · num_output_dim · hidden_dims (encoder)	True · 3 · [256, 128]	—	encoder ตัดจากกราฟหลัก · คิน ๓ ค่า	05 \$4.4
limx_base_env_cfg.py:119-120	scale · use_default_offset	0.25 · True	—	action $\rightarrow$ เรเดียม	05 \$2.5
limx_base_env_cfg.py:497-501	decimation · episode_length_s · sim.dt	4 · 20.0 · 0.005	—	50 Hz policy / 200 Hz physics / 1000 step ต่อ episode	05 \$1.4
cfg/SF/limx_base_env_cfg.py:484	num_envs (ค่าปรัยาย)	4096	2	ถูกกับด้วย CLI --num_envs 2048	05 \$4.7
solefoot_cfg.py:48-62	stiffness · damping · effort_limit	45 · 1.5 (ข) / 0.8 (ข้อเท้า) · 80	—	PD gains	05 \$2.5
ppo.py:61, 63-64	est_learning_rate · early_stop · anneal_lr	1e-3 · False · False	—	ค่าปรัยายของคลาส ไม่ได้ตั้งใน cfg	06 \$3.4

7. ระวังสับสน — ตัวอักษรซ้ำ คำซ้ำ

ตัวอักษร / คำ	ความหมายที่ 1	ความหมายที่ 2 (3)	กฎของไม้ตกนี้
ϵ vs ε vs eps	ϵ = ขอบ clip 0.2	ε = noise มาตรฐาน $\mathcal{N}(0, 1)$ · eps = ตัวกันหารศูนย์ใด ๆ (Adam 1e-8, normalize 1e-8, KL 1e-5)	สามตัวอักษร สามความหมาย · ϵ -greedy เขียนอักษรละติน
T	สมการ (2): ทำสุดท้ายของ episode (ไม่รู้ล่วงหน้า)	หัวข้อ 2 และสมการ (26): จำนวนแถวของตาราง = num_steps_per_env 24 คงที่ · $A_T = 0$ ที่ขอบตาราง	ตัวเดียวกันสองความหมายตามบริบท — ในตาราง 2×4 หมายถึงแถว
r_t vs R_{t+1}	ไม้ตกนี้: reward ของช่อง (s_t, a_t) เรียก r_t ตาม rollout_storage	Sutton & Barto เรียกตัวเดียวกันว่า R_{t+1} (ได้รับหลังทำ)	เวลาเทียบกับตาราง เลื่อนดัชนี 1
L vs ℓ	L = loss, scalar ตัวเดียวทั้งระบบ, เข็ม	ℓ = logstd, เวกเตอร์ 8 ตัว, ลูกบัต	ลูกบัตไปยังเข็ม ไม่ใช่ของชนิดเดียวกัน · L^{CLIP}, L_V = ด้วยกบออกชนิดตัวห้อยนอก network
π	policy	3.14159 ใน $\sqrt{2\pi}$	ดูบริบท — ถ้ามีตัวห้อย θ หรือวงเล็บ คือ policy
π_θ vs $\pi_\theta(\cdot s)$ vs $\pi_\theta(a s)$	ฟังก์ชัน	การแจกแจง · scalar	สามจุดบนสายชนิด — คนละของ
$R(s, a)$ vs G_t vs R_t	ฟังก์ชันรางวัล (คณิต)	return จริง (ข้อเท็จจริง) · returns = $A_t + V_t$ (เป้าหมายของ critic มี critic ใด)	เขียนวงเล็บ / ตัวห้อยเสมอ ห้าม R ลอย ๆ
A vs A_t vs $\hat{A}_t$	เซตของ action	advantage ใด · advantage ที่ normalize (ตัวที่ loss ใช้)	ห้าม A ลอย ๆ
$\Delta(A)$ vs δ_t vs $\Delta\mu$	simplex	TD error · ระยะที่ μ ขยับ	ตัวใหญ่มีวงเล็บ = simplex · ตัวเล็กมีตัวห้อย = TD
$\sigma: \mathcal{N}(\mu, \sigma)$ vs $\mathcal{N}(\mu, \sigma^2)$	ไม้ตกนี้: ช่องสอง = ส่วนเบี่ยงเบนมาตรฐาน (ตาม scale ของ torch)	ตาราง: ช่องสอง = ความแปรปรวน	ทุกครั้งที่ต้องตาราง เติม "(σ^2 ในตาราง)"
z	เดิม = latent ของ encoder	z_i = ะยะมาตรฐาน $(a_i - \mu_i)/\sigma_i$	latent เปลี่ยนชื่อเป็น $\hat{z}$ · z เหลือความหมายเดียว
h	obsHistory (B, 360)	เดิม ขึ้นของผลต่างกลาง	ขึ้นผลต่างเขียน $\Delta\ell \cdot h$ เหลือความหมายเดียว
①②③ vs กิศ vs ช่อง	①②③ = สามขั้นของ policy (NN, logstd, สูตร...)	ทิศทางกางแปลง = กิศ $n / u / k$ · ช่วงของวัตถุในไดอะแกรม = 1–4	สามระบบตัวเลข สามชนิดของสิ่ง
B vs "batch" vs MB	B = num_envs 2048	"batch" เล็กใช้ สำหรับ 49,152 – เรียก rollout · MB = 12,288	
α	learning rate	(ใน Beta(α, β) ของตารางสถิติ)	ไม้ตกนี้ใช้ความหมายแรกเท่านั้น
"step"	policy step 0.02 s	env.step() · optimizer.step() · physics sub-step 0.005 s	ระบุทุกครั้งว่า step ของอะไร
"clip"	ρ clip ที่ [0.8, 1.2]	value clip ที่ ± 0.2 · clip_grad_norm_ ที่ 1.0	สองอันแรกใช้ clip_param ตัวเดียวกัน อันที่สามคนละเลข
"mean"	μ จุดกลาง	.mean() เผลยทั้ง minibatch · Mean reward ในล็อก	
"policy"	ฟังก์ชัน $S \rightarrow \Delta(A)$ (ตาราง)	NN ที่ deploy (โปรแกรมเมอร์) · ไฟล์ policy.onnx	สามความหมายตรงกันตอน deploy เท่านั้น — ตอนเทรน NN เดียว ๆ ยังไม่ใช่ policy

ตัวอักษร / คำ	ความหมายที่ 1	ความหมายที่ 2 (3)	กฎของไม้ขีดนี้
"objective"	J — objective จริง: ผลตอบแทนคาดหวังทั้ง episode ที่สมการ (2) สิ่งให้ทำให้สูงสุด	L^{CLIP} สมการ (29) — surrogate objective: พังค์ชัน ตัวแทนที่ PPO ได้จริงในแต่ละ minibatch เพราะได้ J ตรง ๆ ไม่ได้	ทั้งสองความหมายคือ "ตัวเลขที่อยากให้สูงขึ้น" (ตรงข้าม loss) จึงลบ เครื่องหมายเป็น loss หรือบวกโบนัสเข้าไปได้ · เมื่อหมายถึง (29) โดยเฉพาะ เขียนเต็มว่า surrogate objective
"KL"	ปริมาณ $D_{KL}(p \parallel q)$	ตัวเลข Policy/mean_kl ในลือก · "KL penalty" (พจน์ ใน loss — Tron1 ไม่มี)	
"การแจกแจงบน" vs "ของ"	บน $\mathcal{A}$ (พื้นที่ สุ่มไม่ได้)	ของ $\mathbf{a}$ (ตัวแปรสุ่ม ทอยใหม่ได้)	ถามว่า "สิ่งนี้สุ่มได้ไหม"
ตัวห้อย t	step ในตาราง เริ่ม 0 ใหม่ทุก iteration	(ในตาราง: เวลาต่อเนื่องใน episode)	ไม้ขีดนี้ใช้ความหมายแรก
kl_mean vs mean_kl	ค่าที่คุม lr (ppo.py:233)	ค่าเฉลี่ย 20 sou ที่ขึ้น tensorboard เท่านั้น (: 295)	

8. ตัวอย่างเดียวที่ใช้ทั้งสามไม้ขีด

ตารางจัว 2 env × 4 step, action 1 มิติ — เลิกพอเขียนทุกเลขลงกระดาษ · ไม้ขีด 05 หยิบทีละช่อง ไม้ขีด 06 ขยายเป็น 8 มิติเฉพาะจุดที่ shape สำคัญ ไม้ขีด 07 ใส่ทั้ง iteration · ตัวเลขทุกตัวข้างล่าง คำนวณด้วยสคริปต์เดียวและตรวจซ้ำแล้ว

8.1 ค่าคงที่

ค่าคงที่	toy	Tron1
$B \cdot T$ · มิติของ $\mathbf{a}$	$2 \cdot 4 \cdot 1$	$2048 \cdot 24 \cdot 8$
$\gamma \cdot \lambda \cdot \gamma \lambda$	$0.9 \cdot 0.8 \cdot 0.72$	$0.99 \cdot 0.95 \cdot 0.9405$
σ	0.5 คงที่ ($\ell = -0.6931$)	เริ่ม 1.0 แล้วเรียนรู้
ϵ (clip)	0.2	0.2
epoch × minibatch	$2 \times 2 = 4$ gradient step	$5 \times 4 = 20$
การยับ ม ที่ใช้สารถ	step แรก: $\mu_{new} = \mu_{old} + 0.05$ ทุกช่อง · กรณี clip: $\mu_{new} = \mu_{old} + \text{sign}(\hat{A}) \text{sign}(\mathbf{a} - \mu_{old}) \cdot 0.15$ (ก๊ศที่ (G) สิ่ง)	—

8.2 ข้อมูลดิบ — สิ่งที่ NN คาย (μ, V) · ลูกเต๋าของ policy (ϵ) · สิ่งที่โลกให้มา ($r, done$)

		t=0	t=1	t=2	t=3	V_{last}
env 0	μ	0.20	0.30	0.10	0.40	
	ϵ	+0.40	−0.20	+0.80	0.00	
	r	1.0	0.5	1.0	0.5	
	V	5.0	4.5	4.0	3.5	3.0
	done	0	0	0	0	
env 1	μ	−0.10	0.00	0.20	−0.30	
	ϵ	−0.60	+0.80	+0.20	+0.40	
	r	0.5	1.0	−2.0	0.0	
	V	3.0	2.0	1.0	4.0	3.5
	done	0	0	1	0	

ออกแบบให้ครบทุกกรณี: env 1 ล้มที่ t2 (done ตัดสาย GAE, episode ใหม่เริ่ม t3) · **e0 t2 กับ e1 t1 เป็นคู่แฝด** ($|\mathbf{a} - \mu| = 0.40$ เท่ากัน, $\hat{A}$ คนละเครื่องหมาย) ไว้ดู clip ทั้งสี่ กรณี

8.3 เฉลย — ทุกค่าที่ derive ได้จาก 8.2

ช่อง	$\mathbf{a}$	$\log \pi_{old}$	δ_t	A_t	R_t	$\hat{A}_t$	$\log \pi_{new} (+0.05)$	ρ	L_i	$\partial L / \partial \mu$
e0 t0	0.40	−0.3058	+0.0500	−0.2722	4.7278	+0.7975	−0.2708	1.0356	−0.8259	−0.4955
e0 t1	0.20	−0.2458	−0.4000	−0.4475	4.0525	+0.6452	−0.2708	0.9753	−0.6293	+0.3776
e0 t2	0.50	−0.5458	+0.1500	−0.0660	3.9340	+0.9766	−0.4708	1.0779	−1.0526	−1.4737
e0 t3	0.40	−0.2258	−0.3000	−0.3000	3.2000	+0.7733	−0.2308	0.9950	−0.7695	+0.1539
e1 t0	−0.40	−0.4058	−0.7000	−2.3272	0.6728	−0.9874	−0.4708	0.9371	+0.9253	−1.2954
e1 t1	0.40	−0.5458	−0.1000	−2.2600	−0.2600	−0.9290	−0.4708	1.0779	+1.0014	+1.4020
e1 t2	0.30	−0.2458	−3.0000	−3.0000	−2.0000	−1.5718	−0.2308	1.0151	+1.5955	+0.3191
e1 t3	−0.10	−0.3058	−0.8500	−0.8500	3.1500	+0.2956	−0.2708	1.0356	−0.3062	−0.1837

$\text{mean}(\mathbf{A}) = -1.1904 \cdot \text{std}_{n-1}(\mathbf{A}) = 1.1513 \cdot \text{mb}_0 = \{\text{e0t0, e0t2, e1t1, e1t3}\}$ เฉลี่ย $L^{CLIP} = \mathbf{-0.2958}$ · mb_1 เฉลี่ย = $+0.2805 \cdot \mathbf{H} = 0.7258 \cdot \text{KL}(\Delta \mu = 0.05) = 0.0050$

8.4 การ์ดช่องมาตรฐาน — e0 t0 (ค่าที่คอลัมน์ "ค่าใน toy" ใช้)

ปริมาณ	ค่า	มาจาก
μ, ε, σ	0.20, +0.40, 0.5	ข้อมูลดิบ
$a = \mu + \sigma \varepsilon$	$0.20 + 0.5 \times 0.40 = \mathbf{0.40}$	(16)
$z = (a - \mu) / \sigma$	0.40	
$\pi(a \mid s) \cdot \log \pi$	0.7365 · −0.3058	(13), (14) — ยอด ระฆังที่ $\sigma = 0.5$ คือ 0.7979
$r, V, V(s_1)$	1.0, 5.0, 4.5	ข้อมูลดิบ
Bellman ตรวจ	$1.0 + 0.9 \times 4.5 = 5.05$ vs 5.0	(4) — ส่วนต่างคือ δ
$\delta \cdot A \cdot R \cdot \hat{A}$	$+0.05 \cdot -0.2722 \cdot 4.7278 \cdot +0.7975$	(19), (26), (27), (28)
หลัง μ ขยับ +0.05: $\log \pi_{new} \cdot \rho \cdot L_i \cdot \partial L / \partial \mu$	$-0.2708 \cdot \mathbf{1.0356} \cdot \mathbf{-0.8259} \cdot \mathbf{-0.4955} \rightarrow \text{SGD } \alpha = 0.1: \mu \text{ 0.25 } \rightarrow 0.2995$	(21), (29), (24)
ที่ gradient step แรก ($\rho = 1$): $\partial L / \partial \ell$	surrogate $-\hat{A}(z^2 - 1) = \mathbf{+0.6699}$ · entropy $-0.01 \cdot \text{รวม} \mathbf{+0.6599} \rightarrow$ ถ้าด้วย lr จริงของ Tron1 $\alpha = 10^{-3}$ (ไม่ใช่ 0.1 ของแควบน — เพื่อให้เห็นว่าถ้าจริงเล็กแค่ไหน; ส่วนอื่นของ toy ตรง σ ิ้): $\ell \text{ } -0.6931 \rightarrow -0.693807 \rightarrow \sigma \text{ 0.499670}$	(35)
ถ้า critic ขยับเป็น $V_{new} = 4.5$ (สมมติ): L_V	$V_{clip} = 5.0 + \text{clip}(4.5 - 5.0, \pm 0.2) = 4.8 \cdot \max((4.5 - 4.7278)^2, (4.8 - 4.7278)^2) = \max(0.0519, 0.0052) = 0.0519$	(32)

8.5 ช่องอื่นที่โน้ตหยิบใช้

ช่อง	ใช้ทำอะไร	ค่าสำคัญ
e1 t1 (คู่แฝดของ e0 t2)	action แย้ ที่ระยะเดียวกัน	$a = 0.40, \hat{A} = -0.9290, \log \pi_{old} = -0.5458$
e0 t2	action ดี ที่ระยะเดียวกัน	$a = 0.50, \hat{A} = +0.9766, \log \pi_{old} = -0.5458$
คู่แฝดกับกฎ ± 0.15	clip คบสักรัณ	ขยับ ถูกทาง 0.15: $\rho = 1.2153$ (e0 t2) / 0.7520 (e1 t1) $\rightarrow$ clip, gradient 0 · ขยับ ผิดทาง 0.15: ρ เท่ากันแต่ max เลือกพอนด์ gradient ไหล $-1.6157 / +1.1291 \cdot \text{KL} = 0.045 > 0.02 \rightarrow \text{lr} \div 1.5$
e1 t2 (done = 1)	mask ตัด bootstrap	$\delta = -3.00$ · ถ้าไม่มี mask จะได้ $+0.6$ ("ล้นแล้วดีกว่าที่คาด" — ผิด)
คอลัมน์ env 1	episode ที่จบจริง	$G_0 = -0.22$ vs $V(s_0) = 3.0$ · TD ถ้าเดียว = 2.3
e0 t3	$\varepsilon = 0 \rightarrow a = \mu$	$\log \pi = -0.2258$ = ยอดระฆัง

8.6 การขยายเป็น 8 มิติ (ใช้เฉพาะโน้ต 06 §3.5.5 และ §3.5.7)

ข้อต่อที่ 1 $\equiv$ e0 t0 ส่วนข้อต่อ 2–8 เป็นเลขอิสระ:

$$\begin{aligned} \mu &= [0.20, \text{ } -0.30, 0.10, 0.40, \text{ } -0.10, 0.60, \text{ } -0.50, 0.30] & \varepsilon &= [0.40, \text{ } -0.20, 0.60, 0.00, \text{ } -0.60, 1.20, \text{ } -0.80, 0.20] \\ a &= [0.40, -0.40, 0.40, 0.40, -0.40, 1.20, -0.90, 0.40] \cdot \\ \log \mathcal{N}_i &= [-0.3058, -0.2458, -0.4058, -0.2258, -0.4058, -0.9458, -0.5458, -0.2458] \cdot \Sigma = \mathbf{-3.3263} \cdot \pi = \mathbf{0.03592} \cdot \mu \text{ ขยับ } +0.05 \text{ ทุกมิติ:} \\ \Sigma \log \pi_{new} &= \mathbf{-3.2863} \cdot \rho = 1.0408 \cdot L^{CLIP} = \mathbf{-0.8300} \text{ (ที่ } \hat{A} = +0.7975) \cdot \\ \partial L / \partial \mu_i &= [-0.498, +0.498, -0.830, +0.166, +1.162, -1.826, +1.494, -0.166] \cdot H = \mathbf{5.8063} \cdot \text{KL} = 0.040 \end{aligned}$$

8.7 เลขที่เลิกใช้แล้ว

$$\sigma = 0.3, \pi = 1.088, \rho = 1.22, 0.1525 \text{ rad (โน้ต 05 เดิม)} \cdot \mu = \mathbf{0.42}, \hat{A} = \mathbf{+1.10}, \log \pi = \mathbf{-3.3955}, \rho = \mathbf{1.0953}, 0.03352 \text{ (โน้ต 06 เดิม)} \cdot V = \mathbf{650}, \delta = \mathbf{28.15}, \rho = 1.25, L_V = \mathbf{781.2}$$
 (walkthrough ต้นทาง — เหลือเป็นตัวอย่างขนาดจริงในโน้ต 07 §6 และ errata ในโน้ต 06 §4 เท่านั้น)

9. ดัชนีสมการ

เลขเดียวกันทั้งชุด เรียงตามลำดับที่โน้ต 05 พู · (35)–(37) เป็นของโน้ต 06 · โน้ต 07 ใช้เลขเดียวกัน

#	สมการ	อ่านว่า	derive ที่
(1)	$P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_t, a_t, s_{t-1}, \dots)$	Markov: อดีตไม่เพิ่มข้อมูล	05 §1.3
(2)	$\pi^* = \arg \max_{\pi} \mathbb{E} \big[\sum_{t=0}^T \gamma^t r_t \big]$	โจทย์ทั้งหมด: หา π ที่ทำให้ objective J สูงสุด	05 §1.5
(3)	$V^\pi(s) = \mathbb{E}_{\pi} \big[\sum_{k \geq 0} \gamma^k r_{t+k} \mid s_t = s \big]$	นิยาม value	05 §1.6
(4)	$V^\pi(s) = \mathbb{E} \big[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s \big]$	Bellman	05 §1.6
(5)	$Q^\pi(s, a) = \mathbb{E}_{\pi} \big[\sum_{k \geq 0} \gamma^k r_{t+k} \mid s_t = s, a_t = a \big]$	นิยาม action-value	05 §1.6
(6)	$Q^\pi(s, a) = \mathbb{E} \big[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a \big]$	Q ถ้าเดียว	05 §1.6
(7)	$V^\pi(s) = \mathbb{E} [G_t \mid s_t = s]$	value = ทำนาย return	05 §1.7
(8)	$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$	advantage	05 §1.7
(9)	$\pi_\theta(\cdot \mid s) = \mathcal{N}(\mu_\theta(s), \sigma), \quad \sigma = e^\ell$	policy ของ Tron1	05 §2.1
(10)	$y = f(\sum_i w_i x_i + b)$	หนึ่ง node	05 §2.2
(11)	$\text{ELU}(x) = x \text{ } (x > 0), e^x - 1 \text{ } (x \leq 0)$	activation	05 §2.2

#	สมการ	จำนว	derive ที่
(12)	$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$	gradient descent	05 §2.2
(13)	$\pi(a \mid s) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(a-\mu)^2}{2\sigma^2}\right)$	Gaussian PDF	05 §2.3
(14)	$\log \pi(a \mid s) = -\frac{1}{2} \ln 2\pi - \ln \sigma - \frac{(a-\mu)^2}{2\sigma^2}$	log-density สามพจน์	05 §2.3 · 07 §2.2
(15)	$\log \pi_{\theta}(a \mid s) = \sum_{i=1}^8 \log \pi(a_i \mid s)$	รวม 8 ข้อต่ออิสระ	05 §2.3
(16)	$a = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$	สุ่ม	05 §2.4
(17)	$q_{\text{target}} = 0.25 a + q_{\text{default}}$	action → บุนเป้าหมาย	05 §2.5
(18)	$\tau = k_p(q_{\text{target}} - q) - k_d \dot{q}$	PD	05 §2.5
(19)	$\delta_t = r_t + (1 - \text{done}_t) \gamma V(s_{t+1}) - V(s_t)$	TD error (มี mask)	05 §3.2 · 07 §4.2
(20)	$\mathbb{E}_{a \sim p}[f(a)] = \mathbb{E}_{a \sim q}\left[\frac{p(a)}{q(a)} f(a)\right]$	importance sampling	05 §3.3
(21)	$\rho_t = \frac{\pi_{\text{new}}(a_t s_t)}{\pi_{\text{old}}(a_t s_t)} = \exp(\log \pi_{\text{new}} - \log \pi_{\text{old}})$	ratio	05 §3.3 · 07 §5.2
(22)	$\nabla_{\theta} \pi_{\theta} = \pi_{\theta} \nabla_{\theta} \log \pi_{\theta}$	score function	05 §4.1
(23)	$\nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a \mid s) A(s, a)]$	policy gradient	05 §4.1
(24)	$\partial \log \pi / \partial \mu = (a - \mu) / \sigma^2$	ทศของ μ	05 §4.1 · 07 §5.2
(25)	$A_t = \sum_{k \geq 0} (\gamma \lambda)^k \delta_{t+k}$	GAE พววม	05 §4.5
(26)	$A_t = \delta_t + (1 - \text{done}_t) \gamma \lambda A_{t+1}, A_T = 0$	GAE recursion (โค้ด)	05 §4.5 · 07 §4.3
(27)	$R_t = A_t + V_t$	เป้าของ critic	05 §4.5 · 07 §4.4
(28)	$\hat{A} = (A - \bar{A}) / (\text{std}_{n-1}(A) + 10^{-8})$	normalize	05 §4.5 · 07 §4.5
(29)	$L^{\text{CLIP}} = \text{mean}[\max(-\rho \hat{A}, -\text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) \hat{A})]$	surrogate loss (รูปโค้ด)	05 §4.6 · 07 §5.2
(30)	$D_{KL}(\mathcal{N}_o \parallel \mathcal{N}_n) = \sum_i \left[\ln \frac{\sigma_{n,i}^2}{\sigma_{o,i}^2} + \frac{\sigma_{o,i}^2 + (\mu_{o,i} - \mu_{n,i})^2}{2\sigma_{n,i}^2} - \frac{1}{2} \right]$	KL ของเกาส์เซียน	05 §4.6 · 07 §5.7
(31)	$H = \sum_i \left[\frac{1}{2} + \frac{1}{2} \ln 2\pi + \ln \sigma_i \right]$	entropy	05 §4.6 · 07 §5.5
(32)	$L_V = \text{mean}[\max((V - R)^2, (V_{\text{clip}} - R)^2)], V_{\text{clip}} = V_{\text{old}} + \text{clip}(V - V_{\text{old}}, \pm \epsilon)$	value loss	05 §4.6 · 07 §5.4
(33)	$L = L^{\text{CLIP}} + c_V L_V - c_H H$	loss รวม	05 §4.6 · 07 §5.6
(34)	$1/(1 - \gamma \lambda)$	horizon ที่ GAE มองเห็น	05 §4.7
(35)	$\partial L^{\text{CLIP}} / \partial \ell_i = -\hat{A} \rho (z_i^2 - 1), \quad \partial H / \partial \ell_i = 1$	ทศของ σ	06 §3.4
(36)	$\theta \leftarrow \theta - \alpha \hat{m} / (\sqrt{\hat{v}} + \text{eps})$	Adam	06 §3.4
(37)	$\text{KL} > 2 \cdot \text{desired} \rightarrow \text{lr} \div 1.5 \cdot \text{KL} < \frac{1}{2} \cdot \text{desired} \rightarrow \text{lr} \times 1.5$ ในช่วง [1e-5, 1e-2]	กฎปรับ lr	06 §3.4
(G)	$\partial L / \partial \mu = -\hat{A} \rho (a - \mu_{\text{new}}) / \sigma^2$	(24) ต่อผ่าน p และ (29) — เลขของโค้ด 07	07 §5.2

ตารางแปลงเลขเก่า

เก่า	เลขเก่า → ใหม่
โค้ด 05 เดิม (1)–(12)	(1) → (2) · (2) → (3) · (3) → (4) · (4) → (7) · (5) → (10) · (6) → (12) · (7) → (13) · (8) → (16) · (9) → (18) · (10) → (8) · (11) → (25) · (12) → (29)
โค้ด 07 เดิม	(7') → (14) · (10') → (19) · (11') → (26) · (4') → (27) · (G) คงเดิม
โค้ด 06 ขึ้น E1–E15	E1–E2 ไม่มีสมการ (แต่ tensor) · E3 ↔ (10) · E4 ↔ (9) · E5 ↔ (9) · E6 ↔ (16) · E7 ↔ (13) · E8 ↔ (14) · E9 ↔ (15) · E10–E12 ไม่มีสมการ (เก็บ/อ่าน buffer, ค่ารวมใหม่) · E13 ↔ (21) · E14 ↔ (29) · E15 ↔ (24)/(G)
walkthrough ต้นทาง	(1b) → (12) · (3) → (4) · (5) → (26) · (6) → (29)

ตรวจกับโค้ดจริง 2026-09-11 · ตัวเลขทั้งหมดในหัวข้อ 8 คำนวณด้วย `../scripts/toy_answer_key.py` (รับ `python3 scripts/toy_answer_key.py` จากรากของ repo) · บรรทัดใน `torch/` อ้างจาก torch 2.5.1 ใน env `trn1` — เลขบรรทัดของไลบรารีขึ้นกับเวอร์ชัน